

MongoDB + Java

Guía Completa de Integración

Tipos de datos · Variables de conexión · Métodos · Valores de retorno

Tabla de Contenidos

1. Introducción a MongoDB
2. Dependencia Maven / Gradle
3. Tipos de Datos en MongoDB (BSON)
4. Conexión a MongoDB — MongoClient
5. MongoDB — Base de Datos
6. MongoCollection — Colección
7. Operaciones CRUD Detalladas
8. Filtros y Proyecciones
9. Índices
10. Aggregation Pipeline
11. Transacciones
12. GridFS — Archivos
13. Manejo de Errores
14. Ejemplo Completo de Aplicación

1. Introducción a MongoDB

MongoDB es una base de datos NoSQL orientada a documentos. En lugar de filas y columnas, almacena los datos como **documentos BSON** (Binary JSON) dentro de **colecciones**. El driver oficial de Java (*mongodb-driver-sync*) permite interactuar con MongoDB de forma síncrona utilizando una API fluida basada en builders.

Conceptos clave:

Base de datos	Agrupación de colecciones. Equivale a un schema en SQL.
Colección	Conjunto de documentos. Equivale a una tabla en SQL.
Documento	Unidad de datos en formato BSON (clave-valor). Equivale a una fila en SQL.
_id	Campo obligatorio que identifica de forma única cada documento (por defecto ObjectId).
BSON	Formato binario de serialización que extiende JSON con tipos adicionales.

2. Dependencia Maven / Gradle

Maven (pom.xml)

```
<dependency>
  <groupId>org.mongodb</groupId>
  <artifactId>mongodb-driver-sync</artifactId>
  <version>4.11.1</version>
</dependency>
```

Gradle (build.gradle)

```
implementation 'org.mongodb:mongodb-driver-sync:4.11.1'
```

3. Tipos de Datos en MongoDB (BSON)

BSON define una colección extendida de tipos de datos. A continuación se presenta cada tipo con su clase Java correspondiente y cómo se crea un valor de ese tipo.

Tipo BSON	Clase Java	Ejemplo Java	Descripción
ObjectId	org.bson.types.ObjectId	<code>new ObjectId() / new ObjectId("64abc...")</code>	Identificador único de 12 bytes. Usado como <code>_id</code> por defecto.
String	java.lang.String	<code>"Hola Mundo"</code>	Texto UTF-8.
Integer (32 bit)	java.lang.Integer	<code>42</code>	Entero de 32 bits.
Long (64 bit)	java.lang.Long	<code>123456789L</code>	Entero de 64 bits.
Double	java.lang.Double	<code>3.14</code>	Número de punto flotante IEEE 754.
Decimal128	org.bson.types.Decimal128	<code>Decimal128.parse("19.99")</code>	Decimal de alta precisión (128 bits).
Boolean	java.lang.Boolean	<code>true / false</code>	Valor booleano.
Date	java.util.Date	<code>new Date()</code>	Fecha/hora en milisegundos desde epoch UTC.
Array	java.util.List	<code>Arrays.asList(1, 2, 3)</code>	Lista ordenada de valores (pueden ser de distintos tipos).
Document (obj.)	org.bson.Document	<code>new Document("clave", valor)</code>	Documento embebido (objeto anidado).
Null	null (Java)	<code>null</code>	Valor nulo explícito.
Binary	org.bson.types.Binary	<code>new Binary(bytes[])</code>	Datos binarios arbitrarios.
Regular Expr.	java.util.regex.Pattern	<code>Pattern.compile("^Java")</code>	Expresión regular.
Timestamp	org.bson.BsonTimestamp	<code>new BsonTimestamp(seconds, inc)</code>	Timestamp interno de replicación.
MinKey / MaxKey	BsonMinKey / BsonMaxKey	<code>new BsonMinKey()</code>	Valores especiales para comparaciones en índices.

4. Conexión a MongoDB — MongoClient

MongoClient es el punto de entrada principal al driver. Representa el pool de conexiones y debe ser instanciado **una sola vez** por aplicación (es thread-safe). Implementa *Closeable*, por lo que se puede usar con try-with-resources.

Formas de crear MongoClient

a) Cadena de conexión (URI)

```
import com.mongodb.client.MongoClient;
import com.mongodb.client.MongoClients;

// Local sin autenticación
MongoClient client = MongoClients.create("mongodb://localhost:27017");

// Con usuario y contraseña
MongoClient client = MongoClients.create(
    "mongodb://usuario:contrasena@localhost:27017/miDB?authSource=admin");

// Atlas (cluster en la nube)
MongoClient client = MongoClients.create(
    "mongodb+srv://usuario:pass@cluster0.xxxxx.mongodb.net/?retryWrites=true");
```

b) MongoClientSettings (configuración programática)

```
import com.mongodb.MongoClientSettings;
import com.mongodb.ServerAddress;
import com.mongodb.MongoCredential;
import com.mongodb.ConnectionString;

MongoCredential credential = MongoCredential.createCredential(
    "usuario", // username
    "admin", // authSource (base de datos de autenticación)
    "pass".toCharArray() // password como char[]
);

MongoClientSettings settings = MongoClientSettings.builder()
    .credential(credential)
    .applyToClusterSettings(b ->
        b.hosts(Arrays.asList(new ServerAddress("localhost", 27017))))
    .applyToConnectionPoolSettings(b ->
        b.maxSize(20).minSize(5)) // pool: 5-20 conexiones
    .build();

MongoClient client = MongoClients.create(settings);
```

Métodos de MongoClient

<code>getDatabase(String name)</code>	<code>MongoDatabase</code>	Retorna el objeto de la base de datos con el nombre indicado. No lanza error si no existe: se crea al insertar el primer documento.
<code>listDatabaseNames()</code>	<code>MongoIterable<String></code>	Retorna un iterable con los nombres de todas las bases de datos del servidor.
<code>listDatabases()</code>	<code>ListDatabasesIterable<Document></code>	Retorna información detallada (nombre, tamaño, vacío) de cada base de datos.
<code>startSession()</code>	<code>ClientSession</code>	Inicia una sesión de cliente. Necesaria para transacciones multi-documento.
<code>close()</code>	<code>void</code>	Cierra el pool de conexiones y libera recursos. Llamar al apagar la aplicación.
<code>getClusterDescription()</code>	<code>ClusterDescription</code>	Retorna metadatos del cluster (tipo, servidores, estado de conexión).

5. MongoDB — Base de Datos

MongoDatabase representa una base de datos específica. Se obtiene de MongoClient y es el contenedor de las colecciones.

```
MongoDatabase db = client.getDatabase("miBaseDeDatos");

// Obtener nombre de la BD
String nombre = db.getName(); // => "miBaseDeDatos"
```

Métodos de MongoDB

getName()	String	Retorna el nombre de la base de datos.
getCollection(String name)	MongoCollection<Document>	Retorna la colección con ese nombre (tipo Document).
getCollection(name, Class<T>)	MongoCollection<T>	Retorna la colección mapeada a una clase POJO personalizada.
listCollectionNames()	MongoIterable<String>	Lista los nombres de todas las colecciones en esta BD.
listCollections()	ListCollectionsIterable<Document>	Lista info detallada de cada colección (nombre, opciones, etc.).
createCollection(String name)	void	Crea una colección de forma explícita (opciones por defecto).
createCollection(name, options)	void	Crea una colección con opciones como tamaño máximo (capped), validación de esquema, etc.
drop()	void	Elimina TODA la base de datos y sus colecciones. Irreversible.
runCommand(Document cmd)	Document	Ejecuta un comando de administración de MongoDB (ej: {ping:1}, {serverStatus:1}).
createView(name, on, pipeline)	void	Crea una vista basada en una colección y un pipeline de aggregation.
withCodecRegistry(registry)	MongoDatabase	Retorna una copia de la BD con un CodecRegistry personalizado (para mapeo POJO).

Ejemplo: Crear colección con validación

```
CreateCollectionOptions opts = new CreateCollectionOptions()
.validationOptions(new ValidationOptions()
.validator(Filters.exists("email")) // el campo email debe existir
.validationLevel(ValidationLevel.STRICT)
.validationAction(ValidationAction.ERROR));

db.createCollection("usuarios", opts);
```

6. MongoClient — Colección

MongoCollection<T> es la clase más usada del driver. Proporciona todos los métodos CRUD. El tipo genérico *T* puede ser **Document** (acceso genérico) o cualquier clase Java (acceso tipado con POJOs).

```
// Colección genérica
MongoCollection<Document> col = db.getCollection("productos");

// Colección tipada (requiere CodecRegistry con PojoCodecProvider)
MongoCollection<Producto> col = db.getCollection("productos", Producto.class);
```


7. Operaciones CRUD Detalladas

7.1 INSERT — Insertar Documentos

<code>insertOne(T document)</code>	<code>InsertOneResult</code>	Inserta un documento. Retorna <code>InsertOneResult</code> con el <code>_id</code> generado.
<code>insertOne(session, T document)</code>	<code>InsertOneResult</code>	Versión con sesión de cliente (para transacciones).
<code>insertMany(List<T> docs)</code>	<code>InsertManyResult</code>	Inserta múltiples documentos. Por defecto en orden; si uno falla el resto no se inserta.
<code>insertMany(docs, InsertManyOptions)</code>	<code>InsertManyResult</code>	Con opciones: <code>ordered=false</code> permite continuar aunque falle algún documento.

InsertOneResult — Métodos de retorno

<code>getInsertedId()</code>	<code>BsonValue</code>	El <code>_id</code> del documento insertado (puede ser <code>ObjectId</code> , <code>String</code> , etc.).
<code>wasAcknowledged()</code>	<code>boolean</code>	True si el servidor confirmó la escritura (según <code>WriteConcern</code>).

InsertManyResult — Métodos de retorno

<code>getInsertedIds()</code>	<code>Map<Integer, BsonValue></code>	Mapa índice-posición → <code>_id</code> de cada documento insertado.
<code>wasAcknowledged()</code>	<code>boolean</code>	True si el servidor confirmó la escritura.

Ejemplo insertOne e insertMany

```
// insertOne
Document prod = new Document("nombre", "Laptop")
    .append("precio", 999.99)
    .append("stock", 50)
    .append("fecha", new Date());

InsertOneResult r1 = col.insertOne(prod);
BsonValue idInsertado = r1.getInsertedId(); // => BsonObjectId
boolean ok = r1.wasAcknowledged(); // => true

// insertMany
List<Document> lista = Arrays.asList(
    new Document("nombre", "Mouse").append("precio", 25.0),
    new Document("nombre", "Teclado").append("precio", 45.0)
);
InsertManyResult r2 = col.insertMany(lista);
Map<Integer, BsonValue> ids = r2.getInsertedIds(); // {0=ObjectId..., 1=ObjectId...}
```

7.2 FIND — Buscar Documentos

<code>find()</code>	<code>FindIterable<T></code>	Retorna todos los documentos de la colección.
<code>find(Bson filter)</code>	<code>FindIterable<T></code>	Retorna documentos que cumplan el filtro.
<code>find(session, filter)</code>	<code>FindIterable<T></code>	Versión con sesión de cliente.
<code>find(filter, Class<R>)</code>	<code>FindIterable<R></code>	Retorna los resultados mapeados a la clase R.

FindIterable — Métodos encadenables

<code>filter(Bson f)</code>	<code>FindIterable<T></code>	Aplica/reemplaza el filtro de búsqueda.
<code>projection(Bson p)</code>	<code>FindIterable<T></code>	Especifica qué campos incluir o excluir.
<code>sort(Bson sort)</code>	<code>FindIterable<T></code>	Ordena los resultados. Usar <code>Sorts.ascending/descending</code> .
<code>limit(int n)</code>	<code>FindIterable<T></code>	Limita el número máximo de documentos retornados.
<code>skip(int n)</code>	<code>FindIterable<T></code>	Omite los primeros n documentos (paginación).
<code>first()</code>	<code>T</code> (o <code>null</code>)	Retorna solo el primer documento que coincide, o <code>null</code> si no hay.
<code>into(List<T>)</code>	<code>List<T></code>	Recoge todos los resultados en la lista proporcionada.
<code>iterator()</code>	<code>MongoCursor<T></code>	Retorna un cursor para iterar manualmente (cerrar con <code>close()</code>).
<code>forEach(Consumer<T>)</code>	<code>void</code>	Itera y aplica una acción a cada documento.
<code>hint(Bson hint)</code>	<code>FindIterable<T></code>	Fuerza el uso de un índice específico.
<code>maxTime(long, TimeUnit)</code>	<code>FindIterable<T></code>	Establece tiempo máximo de ejecución de la consulta.
<code>collation(Collation c)</code>	<code>FindIterable<T></code>	Define reglas de comparación de texto (locale, mayúsculas, etc.).
<code>explain()</code>	<code>Document</code>	Retorna el plan de ejecución de la query (útil para optimización).

Ejemplo completo de find

```
import com.mongodb.client.model.Filters;
import com.mongodb.client.model.Projections;
import com.mongodb.client.model.Sorts;

// Buscar productos con precio entre 10 y 500, ordenados por precio DESC
// Proyección: solo nombre y precio (excluir _id)
List<Document> resultados = new ArrayList<>();

col.find(Filters.and(
    Filters.gte("precio", 10),
    Filters.lte("precio", 500)))
    .projection(Projections.fields(
        Projections.include("nombre", "precio"),
        Projections.excludeId()))
    .sort(Sorts.descending("precio"))
    .limit(20)
    .skip(0)
    .into(resultados);

// Iterar manualmente con cursor
try (MongoCursor<Document> cursor = col.find().iterator()) {
    while (cursor.hasNext()) {
        Document doc = cursor.next();
        System.out.println(doc.getString("nombre"));
    }
}
```

7.3 UPDATE — Actualizar Documentos

<code>updateOne(filter, update)</code>	<code>UpdateResult</code>	Actualiza el PRIMER documento que coincida con el filtro.
<code>updateOne(filter, update, opts)</code>	<code>UpdateResult</code>	Con opciones adicionales (ej: <code>upsert=true</code>).
<code>updateMany(filter, update)</code>	<code>UpdateResult</code>	Actualiza TODOS los documentos que coincidan.
<code>updateMany(filter, update, opts)</code>	<code>UpdateResult</code>	Con <code>UpdateOptions</code> (<code>upsert</code> , etc.).
<code>replaceOne(filter, replacement)</code>	<code>UpdateResult</code>	Reemplaza COMPLETAMENTE el primer documento coincidente (excepto <code>_id</code>).
<code>findOneAndUpdate(filter, update)</code>	<code>T (o null)</code>	Atomicamente encuentra y actualiza. Retorna el documento ANTES de actualizar.
<code>findOneAndUpdate(f, u, opts)</code>	<code>T (o null)</code>	Con <code>FindOneAndUpdateOptions</code> : <code>returnDocument=AFTER</code> retorna el documento DESPUÉS de actualizar.
<code>findOneAndReplace(f, replacement)</code>	<code>T (o null)</code>	Atomicamente encuentra y reemplaza. Retorna el doc anterior (o nuevo con opción).

UpdateResult — Métodos de retorno

<code>getMatchedCount()</code>	<code>long</code>	Número de documentos que coincidieron con el filtro.
<code>getModifiedCount()</code>	<code>long</code>	Número de documentos realmente modificados.
<code>getUpsertedId()</code>	<code>BsonValue (o null)</code>	Si se hizo <code>upsert</code> , el <code>_id</code> del nuevo documento; <code>null</code> si no.
<code>wasAcknowledged()</code>	<code>boolean</code>	<code>True</code> si el servidor confirmó la operación.

Operadores de actualización (Updates helper)

`Updates.set(campo, valor)` — Establece el valor de un campo (lo crea si no existe).

`Updates.unset(campo)` — Elimina el campo del documento.

`Updates.inc(campo, n)` — Incrementa (o decrementa si `n` es negativo) el valor numérico del campo.

`Updates.push(campo, valor)` — Agrega un elemento al array del campo.

`Updates.addToSet(campo, valor)` — Agrega un elemento al array solo si no existe ya (sin duplicados).

`Updates.pull(campo, valor)` — Elimina todas las ocurrencias de valor en el array.

`Updates.rename(campoViejo, nuevo)` — Renombra un campo.

`Updates.mul(campo, factor)` — Multiplica el valor numérico del campo por el factor dado.

`Updates.min(campo, valor)` — Actualiza el campo solo si valor es menor que el actual.

`Updates.max(campo, valor)` — Actualiza el campo solo si valor es mayor que el actual.

`Updates.currentDate(campo)` — Establece el campo a la fecha/hora actual del servidor.

`Updates.combine(upd1, upd2, ...)` — Combina múltiples operadores de actualización en uno solo.

Ejemplo de updateOne y findOneAndUpdate

```
import com.mongodb.client.model.Updates;
import com.mongodb.client.model.FindOneAndUpdateOptions;
import com.mongodb.client.model.ReturnDocument;

// Incrementar stock y actualizar fecha
UpdateResult ur = col.updateOne(
    Filters.eq("nombre", "Laptop"),
    Updates.combine(
        Updates.inc("stock", 10),
        Updates.currentDate("ultimaActualizacion")
    ));
System.out.println("Modificados: " + ur.getModifiedCount());

// findOneAndUpdate: retornar el documento YA actualizado
FindOneAndUpdateOptions opts = new FindOneAndUpdateOptions()
    .returnDocument(ReturnDocument.AFTER)
    .upsert(false);

Document docActualizado = col.findOneAndUpdate(
    Filters.eq("_id", miId),
    Updates.set("precio", 899.99),
    opts
); // => documento con precio=899.99
```

7.4 DELETE — Eliminar Documentos

<code>deleteOne(Bson filter)</code>	<code>DeleteResult</code>	Elimina el PRIMER documento que coincida con el filtro.
<code>deleteOne(session, filter)</code>	<code>DeleteResult</code>	Versión con sesión (transacciones).
<code>deleteMany(Bson filter)</code>	<code>DeleteResult</code>	Elimina TODOS los documentos que coincidan.
<code>deleteMany(session, filter)</code>	<code>DeleteResult</code>	Versión con sesión.
<code>findOneAndDelete(Bson filter)</code>	<code>T (o null)</code>	Elimina atómicamente y retorna el documento eliminado.
<code>findOneAndDelete(f, options)</code>	<code>T (o null)</code>	Con <code>FindOneAndDeleteOptions</code> : se puede aplicar sort para elegir cuál de varios coincidentes eliminar.
<code>drop()</code>	<code>void</code>	Elimina TODA la colección (documentos e índices).

DeleteResult — Métodos de retorno

<code>getDeletedCount()</code>	<code>long</code>	Número de documentos eliminados.
<code>wasAcknowledged()</code>	<code>boolean</code>	True si el servidor confirmó la operación de escritura.

Ejemplo de delete

```
// Eliminar un documento por _id
DeleteResult dr = col.deleteOne(Filters.eq("_id", new ObjectId("64abc...")));
System.out.println("Eliminados: " + dr.getDeletedCount()); // => 1

// Eliminar todos los productos sin stock
DeleteResult dr2 = col.deleteMany(Filters.lte("stock", 0));
System.out.println("Eliminados: " + dr2.getDeletedCount());

// Obtener el doc antes de eliminarlo
Document eliminado = col.findOneAndDelete(Filters.eq("nombre", "Obsoleto"));
// eliminado puede ser null si no encontró
```

8. Filtros y Proyecciones

Filtros (clase `Filters`)

`Filters.eq(campo, valor)` — Igual a (=)

`Filters.ne(campo, valor)` — Diferente de (!=)

`Filters.gt(campo, valor)` — Mayor que (>)

`Filters.gte(campo, valor)` — Mayor o igual (>=)

`Filters.lt(campo, valor)` — Menor que (<)

`Filters.lte(campo, valor)` — Menor o igual (<=)

`Filters.in(campo, val1, val2, ...)` — Campo cuyo valor está en la lista dada

`Filters.nin(campo, val1, val2, ...)` — Campo cuyo valor NO está en la lista

`Filters.and(f1, f2, ...)` — Conjunción lógica (AND) de varios filtros

`Filters.or(f1, f2, ...)` — Disyunción lógica (OR) de varios filtros

`Filters.not(filtro)` — Negación lógica (NOT)

`Filters.nor(f1, f2, ...)` — Ninguno de los filtros dados

`Filters.exists(campo, true/false)` — El campo existe (o no) en el documento

`Filters.type(campo, BSONType.XXX)` — El campo tiene el tipo BSON indicado

`Filters.regex(campo, pattern)` — El campo coincide con la expresión regular

`Filters.text(textoBuscado)` — Búsqueda de texto completo (requiere índice text)

`Filters.where(jsFunction)` — Expresión JavaScript del lado del servidor

`Filters.elemMatch(campo, filtro)` — Al menos un elemento del array cumple el filtro

`Filters.size(campo, n)` — El array del campo tiene exactamente n elementos

`Filters.all(campo, val1, val2, ...)` — El array contiene todos los valores dados

`Filters.bitsAllSet(campo, bitmask)` — Bits de la máscara están todos activos en el campo

`Filters.geoWithin(campo, geometry)` — Documento dentro de la geometría GeoJSON dada

`Filters.near(campo, lon, lat, maxDist)` — Documentos cercanos a un punto (requiere índice 2dsphere)

`Filters.empty()` — Sin filtro; coincide con todos los documentos

Proyecciones (clase `Projections`)

<code>Projections.include(campo1, campo2, ...)</code>	Incluye solo los campos especificados.
<code>Projections.exclude(campo1, campo2, ...)</code>	Excluye los campos especificados.
<code>Projections.excludeId()</code>	Atajo para excluir el campo <code>_id</code> .
<code>Projections.fields(p1, p2, ...)</code>	Combina múltiples directivas de proyección.
<code>Projections.elemMatch(campo, filtro)</code>	Proyecta el primer elemento del array que cumpla el filtro.
<code>Projections.metaTextScore(campo)</code>	Incluye el score de relevancia en búsquedas de texto.

Projections.slice(campo, n)	Proyecta los primeros n (o últimos si n<0) elementos de un array.
Projections.computed(nombre, expr)	Agrega un campo calculado con una expresión de aggregation.

9. Índices

Los índices aceleran las consultas pero incrementan el uso de disco y ralentizan las escrituras. Se crean con los métodos **createIndex** / **createIndexes** de `MongoCollection`.

<code>createIndex(Bson keys)</code>	<code>String</code>	Crea un índice con las claves dadas. Retorna el nombre del índice.
<code>createIndex(Bson keys, IndexOptions opts)</code>	<code>String</code>	Crea índice con opciones (unique, sparse, expireAfterSeconds, etc.).
<code>createIndexes(List<IndexModel>)</code>	<code>List<String></code>	Crea múltiples índices en una sola operación. Retorna lista de nombres.
<code>listIndexes()</code>	<code>ListIndexesIterable<Document></code>	Lista todos los índices de la colección con sus propiedades.
<code>dropIndex(String name)</code>	<code>void</code>	Elimina el índice con ese nombre.
<code>dropIndex(Bson keys)</code>	<code>void</code>	Elimina el índice que corresponde a esas claves.
<code>dropIndexes()</code>	<code>void</code>	Elimina TODOS los índices excepto el de <code>_id</code> .

Tipos de índice con Indexes helper

```
import com.mongodb.client.model.Indexes;
import com.mongodb.client.model.IndexOptions;

// Índice ascendente simple
col.createIndex(Indexes.ascending("email"));

// Índice único
col.createIndex(Indexes.ascending("email"),
new IndexOptions().unique(true));

// Índice compuesto
col.createIndex(Indexes.compoundIndex(
Indexes.ascending("categoria"),
Indexes.descending("precio")));

// Índice de texto (full-text search)
col.createIndex(Indexes.text("descripcion"));

// Índice TTL (auto-expira documentos)
col.createIndex(Indexes.ascending("creadoEn"),
new IndexOptions().expireAfter(30L, TimeUnit.DAYS));

// Índice geoespacial 2dsphere
col.createIndex(Indexes.geo2dsphere("ubicacion"));

// Índice sparse (solo indexa docs que tienen el campo)
col.createIndex(Indexes.ascending("campoOpcional"),
new IndexOptions().sparse(true));
```

10. Aggregation Pipeline

El pipeline de aggregation procesa documentos en etapas secuenciales. Cada etapa transforma la colección de documentos. Se usa con el método **aggregate()** de `MongoCollection`.

<code>aggregate(List<Bson> pipeline)</code>	<code>AggregateIterable<Document></code>	Ejecuta el pipeline y retorna un iterable de documentos resultantes.
<code>aggregate(session, pipeline)</code>	<code>AggregateIterable<Document></code>	Versión con sesión de cliente.
<code>aggregate(pipeline, Class<R>)</code>	<code>AggregateIterable<R></code>	Mapa resultados a la clase R.

Etapas de Aggregation (clase *Aggregates*)

- `Aggregates.match(filter)` — Filtra documentos según un filtro (igual que `find`).
- `Aggregates.group(id, accumulators...)` — Agrupa documentos por el campo `id` y aplica acumuladores.
- `Aggregates.project(projection)` — Selecciona/transforma campos.
- `Aggregates.sort(sort)` — Ordena los documentos.
- `Aggregates.limit(n)` — Limita la salida a `n` documentos.
- `Aggregates.skip(n)` — Salta los primeros `n` documentos.
- `Aggregates.unwind(campo)` — Descompone un array: genera un doc por cada elemento.
- `Aggregates.lookup(from, local, foreign, as)` — JOIN con otra colección.
- `Aggregates.addFields(Field...)` — Agrega nuevos campos calculados.
- `Aggregates.replaceRoot(expr)` — Reemplaza el documento raíz con un subdocumento.
- `Aggregates.count(campo)` — Cuenta el número de docs y lo guarda en el campo.
- `Aggregates.out(colección)` — Escribe el resultado en una colección (sobreescribe).
- `Aggregates.merge(MergeOptions)` — Mezcla/inserta resultados en una colección existente.
- `Aggregates.facet(Facet...)` — Ejecuta múltiples sub-pipelines en paralelo sobre los mismos datos.
- `Aggregates.bucket(campo, límites...)` — Clasifica documentos en rangos (buckets) definidos.
- `Aggregates.sample(n)` — Selecciona `n` documentos al azar.

Acumuladores (clase *Accumulators*)

- `Accumulators.sum(nombre, expr)` — Suma de los valores de la expresión.
- `Accumulators.avg(nombre, expr)` — Promedio de los valores.
- `Accumulators.min(nombre, expr)` — Valor mínimo.
- `Accumulators.max(nombre, expr)` — Valor máximo.
- `Accumulators.first(nombre, expr)` — Primer valor del grupo.
- `Accumulators.last(nombre, expr)` — Último valor del grupo.
- `Accumulators.push(nombre, expr)` — Crea un array con todos los valores del grupo.
- `Accumulators.addToSet(n, expr)` — Array sin duplicados con todos los valores.
- `Accumulators.count(nombre)` — Cuenta documentos en el grupo.

`Accumulators.stdDevPop(n, expr)` — Desviación estándar de población.

`Accumulators.stdDevSamp(n, expr)` — Desviación estándar muestral.

Ejemplo: ventas por categoría con promedio de precio

```
import com.mongodb.client.model.Aggregates;
import com.mongodb.client.model.Accumulators;

List<Document> resultados = new ArrayList<>();

col.aggregate(Arrays.asList(
    Aggregates.match(Filters.gt("precio", 0)),
    Aggregates.group(
        "$categoria",
        Accumulators.sum("totalVentas", "$ventas"),
        Accumulators.avg("precioPromedio", "$precio"),
        Accumulators.count("numProductos")
    ),
    Aggregates.sort(Sorts.descending("totalVentas")),
    Aggregates.limit(5)
)).into(resultados);

// Cada documento: {_id: "Electrónica", totalVentas: 5000, precioPromedio: 250.0, numProductos: 20}
```

11. Transacciones

MongoDB soporta transacciones ACID multi-documento a partir de la versión 4.0 (replica sets) y 4.2 (sharded clusters). Requieren una **ClientSession**.

```
ClientSession session = client.startSession();

TransactionOptions txOptions = TransactionOptions.builder()
    .readPreference(ReadPreference.primary())
    .readConcern(ReadConcern.LOCAL)
    .writeConcern(WriteConcern.MAJORITY)
    .build();

try {
    session.withTransaction(() -> {
        // Todas estas operaciones son atómicas
        colInventario.updateOne(session,
            Filters.eq("productoId", pid),
            Updates.inc("stock", -1));

        colPedidos.insertOne(session,
            new Document("productoId", pid)
                .append("cantidad", 1)
                .append("fecha", new Date()));

        return null; // commit automático al retornar
    }, txOptions);
} catch (MongoException e) {
    // Si lanza excepción, la transacción se hace ROLLBACK automáticamente
    e.printStackTrace();
} finally {
    session.close();
}
```

Métodos de ClientSession

startSession()	ClientSession	Inicia la sesión de cliente.
withTransaction(body)	T	Ejecuta el body en una transacción. Hace commit si termina bien, rollback si lanza excepción.
startTransaction()	void	Inicia la transacción manualmente.
commitTransaction()	void	Hace commit manual de la transacción activa.
abortTransaction()	void	Hace rollback de la transacción activa.
hasActiveTransaction()	boolean	True si hay una transacción activa en la sesión.
close()	void	Cierra la sesión y libera recursos.

12. GridFS — Archivos Grandes

GridFS divide archivos mayores de 16 MB en chunks de 255 KB y los almacena en dos colecciones: *fs.files* (metadatos) y *fs.chunks* (datos binarios).

```
import com.mongodb.client.gridfs.GridFSBucket;
import com.mongodb.client.gridfs.GridFSBuckets;
import com.mongodb.client.gridfs.model.GridFSUploadOptions;

GridFSBucket gridFS = GridFSBuckets.create(db);
// O con nombre de bucket personalizado:
GridFSBucket gridFS = GridFSBuckets.create(db, "archivos");

// === SUBIR UN ARCHIVO ===
try (InputStream stream = new FileInputStream("video.mp4")) {
    GridFSUploadOptions opts = new GridFSUploadOptions()
        .chunkSizeBytes(358400)
        .metadata(new Document("tipo", "video"));

    ObjectId fileId = gridFS.uploadFromStream("video.mp4", stream, opts);
    System.out.println("Subido con id: " + fileId);
}

// === DESCARGAR UN ARCHIVO ===
try (OutputStream out = new FileOutputStream("descargado.mp4")) {
    gridFS.downloadToStream(fileId, out);
}

// === BUSCAR ARCHIVOS ===
gridFS.find(Filters.eq("filename", "video.mp4"))
    .forEach(f -> System.out.println(f.getFilename() + " - " + f.getLength()));

// === ELIMINAR ===
gridFS.delete(fileId);
```

13. Manejo de Errores

Todas las excepciones del driver heredan de **MongoException** (que extiende RuntimeException). No es necesario declararlas con throws, pero sí capturarlas para una gestión robusta.

MongoException	Base de todas las excepciones del driver MongoDB.
MongoClientException	Error al crear o configurar el MongoClient.
MongoTimeoutException	Tiempo de espera agotado al conectar o ejecutar una operación.
MongoCommandException	El servidor rechazó el comando. Tiene getErrorCode() y getErrorMessage().
MongoBulkWriteException	Uno o más documentos fallaron en insertMany. Tiene getWriteErrors().
MongoWriteException	Fallo en una operación de escritura individual. Ej: violación de índice único.
MongoQueryException	Error en la ejecución de una consulta.
MongoSocketException	Error de red / socket al comunicarse con el servidor.
MongoNotPrimaryException	Se intentó escribir en un secundario de un replica set.
MongoTransactionException	Error durante una transacción multi-documento.
CodecConfigurationException	Fallo en la configuración de codecs para mapeo POJO.

Ejemplo de manejo de errores

```
try {
    col.insertOne(new Document("email", "dup@test.com"));
} catch (MongoWriteException e) {
    if (e.getError().getCategory() == ErrorCategory.DUPLICATE_KEY) {
        System.out.println("Email duplicado: " + e.getMessage());
    }
} catch (MongoCommandException e) {
    System.out.println("Código error: " + e.getErrorCode());
    System.out.println("Mensaje: " + e.getErrorMessage());
} catch (MongoTimeoutException e) {
    System.out.println("Timeout al conectar: " + e.getMessage());
} catch (MongoException e) {
    System.out.println("Error MongoDB: " + e.getMessage());
}
```

14. Ejemplo Completo de Aplicación

A continuación se presenta una clase Java completa que integra todos los conceptos: conexión, CRUD, aggregation y cierre de recursos.

```
import com.mongodb.client.*;
import com.mongodb.client.model.*;
import com.mongodb.client.result.*;
import org.bson.Document;
import org.bson.types.ObjectId;
import java.util.*;

public class MongoDemo {
    public static void main(String[] args) {
        // 1. Conexión
        try (MongoClient client = MongoClient.create("mongodb://localhost:27017")) {
            MongoDBDatabase db = client.getDatabase("tienda");
            MongoCollection<Document> col = db.getCollection("productos");

            // 2. Índice único en 'sku'
            col.createIndex(Indexes.ascending("sku"), new IndexOptions().unique(true));

            // 3. INSERT
            InsertOneResult r = col.insertOne(
                new Document("sku", "A001")
                    .append("nombre", "Monitor 4K")
                    .append("precio", 450.0)
                    .append("stock", 30)
                    .append("tags", Arrays.asList("tech", "display"))
            );
            System.out.println("Insertado: " + r.getInsertedId());

            // 4. FIND
            Document found = col.find(Filters.eq("sku", "A001")).first();
            System.out.println("Encontrado: " + found.getString("nombre"));
        }
    }
}
```



```

// 5. UPDATE
UpdateResult ur = col.updateOne(
    Filters.eq("sku", "A001"),
    Updates.combine(
        Updates.set("precio", 399.99),
        Updates.inc("stock", -5)));
System.out.println("Modificados: " + ur.getModifiedCount());

// 6. AGGREGATE
List<Document> agg = new ArrayList<>();
col.aggregate(Arrays.asList(
    Aggregates.match(Filters.gt("stock", 0)),
    Aggregates.group("$tags",
        Accumulators.avg("precioMedio", "$precio"),
        Accumulators.sum("totalStock", "$stock")),
    Aggregates.sort(Sorts.descending("totalStock"))
)).into(agg);
agg.forEach(d -> System.out.println(d.toJson()));

// 7. DELETE
DeleteResult dr = col.deleteOne(Filters.eq("sku", "A001"));
System.out.println("Eliminados: " + dr.getDeletedCount());

} // client.close() automático (try-with-resources)
}
}

```

Esta guía cubre el driver síncrono *mongodb-driver-sync 4.x*. Para operaciones reactivas consulta **mongodb-driver-reactivestreams**. Documentación oficial: docs.mongodb.com/drivers/java